CO5 – AT1: Constraint-Based Problem Solving

Name:A.Rakesh

Reg No:192525388

Course: Database Management Systems (CSA0504)

Q1. Query Execution Plan & Heuristic Optimization (BL4)

Consider the query:

```
SELECT S.sname, C.cname
FROM   Student S, Enrolled EN, Course C
WHERE  S.sid = EN.sid
  AND  EN.cid = C.cid
  AND  S.dept = 'CSE'
  AND  C.credits > 3
  AND  EN.grade IN (SELECT grade FROM GradeScale WHERE points >= 8)
```

Step 1 — Initial (canonical) parse tree

Every SQL query is first translated into an unoptimized relational-algebra tree: a single large Cartesian product, followed by one big selection (all WHERE conditions ANDed together), followed by the projection. The subquery is represented as a nested sub-tree.

```
π(S.sname, C.cname)
  σ(S.dept='CSE' ∧ C.credits>3 ∧ EN.grade IN (…))
    ×(S, EN, C)
       [subquery] π(grade) σ(points>=8) (GradeScale)
```

Step 2 — Apply heuristic rules

1. Break conjunctive selections into a cascade of individual σ operations, e.g. σ(S.dept='CSE'), σ(C.credits>3), σ(EN.grade IN …), so each can move independently.

2. Push selections down as far as possible toward the leaf relations (commute σ past ×/⋈). σ(S.dept='CSE') moves next to Student; σ(C.credits>3) moves next to Course.

3. Replace the IN sub-query with a semi-join: EN ⋉ (σ(points>=8)(GradeScale)) and push it down next to Enrolled, so the subquery is evaluated once, early, instead of per outer row.

4. Combine each Cartesian product with the selection that makes it an equi-join: × + σ(S.sid=EN.sid) → S ⋈ EN; × + σ(EN.cid=C.cid) → (S⋈EN) ⋈ C.

5. Perform projections early: project out unneeded columns of S, EN, C immediately after the base relations are read, so only sid, sname, dept (for S) and similar minimal attribute sets flow upward.

6. Reorder the joins so that the join producing the smallest intermediate result executes first — here S (filtered on dept) ⋈ EN (filtered by the grade semi-join) is likely much smaller than joining with C first, so it is evaluated before joining with Course.

7. Identify common sub-expressions (e.g. if the GradeScale filter is reused elsewhere in the query) and compute them once.

Step 3 — Optimized execution-plan tree

π(sname, cname)

⋈ (EN.cid = C.cid)

⋈ (S.sid = EN.sid)

σ(dept='CSE') EN ⋉ σ(points>=8)(GradeScale)

Student Enrolled

σ(credits>3)

Course

This heuristically optimized tree touches far fewer tuples at each stage than the naive plan, because selections and the semi-join filter shrink the relations before the expensive joins are performed, and only the required columns are carried forward.

Q2. Cost-Based Optimization — Choosing a Join Algorithm (BL4)

Given relations R and S and M available buffer pages:

Relation	Tuples (n)	Tuples/page	Pages (b)
R	10,000	10	1,000
S	5,000	10	500

Available memory: M = 22 buffer pages.

(a) Block Nested-Loop Join (smaller relation S as outer)

Cost = b(S) + ⌈b(S) / (M-2)⌉ × b(R)

= 500 + ⌈500/20⌉ × 1000 = 500 + 25×1000 = 25,500 I/Os

(b) Sort-Merge Join

External merge-sort cost ≈ 2·b·(1 + passes), where passes = ⌈log(M-1)(b/M)⌉.

Sort R: runs = ⌈1000/22⌉ = 46 → merge passes = ⌈log21(46)⌉ = 2

cost(R) = 2×1000×(1+2) = 6,000

Sort S: runs = ⌈500/22⌉ = 23 → merge passes = ⌈log21(23)⌉ = 1

cost(S) = 2×500×(1+1) = 2,000

Final merge = b(R)+b(S) = 1,500

Total SMJ cost = 6,000 + 2,000 + 1,500 = 9,500 I/Os

(c) Hash Join (Grace / 2-pass hash join)

Feasible when M > √(min(b(R),b(S))); here √500 ≈ 22.4 ≈ M, so a 2-pass hash join is (marginally) usable:

Cost ≈ 3 × (b(R) + b(S)) = 3 × 1,500 = 4,500 I/Os

Decision

Algorithm	**I/O Cost**
Block Nested-Loop	25,500
Sort-Merge Join	9,500
Hash Join	4,500 (cheapest)

The optimizer selects Hash Join because it has the lowest estimated I/O cost given the available memory. If M were too small to satisfy M > √(min(b(R),b(S))) — e.g. fewer than ~22 buffer pages — the optimizer would fall back to Sort-Merge Join instead, since a hash join that must recursively partition adds extra passes and can become more expensive than SMJ.

Q3. JDBC Connectivity with PreparedStatement (BL3)

Java program that connects to MySQL and executes a parameterized query safely (prevents SQL injection):

```
import java.sql.*;
```

```
public class EmployeeQuery {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/company_db";
        String user = "root";
        String password = "your_password";

        String sql = "SELECT emp_id, ename, salary FROM Employee " +
                     "WHERE dept_id = ? AND salary > ?";

        try (Connection conn = DriverManager.getConnection(url, user, password);
             PreparedStatement pstmt = conn.prepareStatement(sql)) {

            pstmt.setInt(1, 10);          // dept_id = 10
            pstmt.setDouble(2, 50000.0);  // salary > 50000

            try (ResultSet rs = pstmt.executeQuery()) {
                while (rs.next()) {
                    System.out.println(rs.getInt("emp_id") + " " +
                        rs.getString("ename") + " " + rs.getDouble("salary"));
                }
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Key points: Class.forName() is not required with modern JDBC 4+ drivers (auto-registered via ServiceLoader); the try-with-resources block auto-closes the Connection, PreparedStatement, and ResultSet; placeholders (?) are bound with setInt/setDouble/setString, which escapes input and prevents SQL-injection attacks, unlike string-concatenated Statement queries.

Q4. Conflict-Serializability via Precedence Graph (BL4)

Schedule S with four transactions:

1. R1(A) 2. R2(B) 3. W1(B) 4. R3(A)

5. W2(A) 6. R4(A) 7. W3(A) 8. W4(B)

Step 1 — Identify conflicting operation pairs

Two operations conflict if they belong to different transactions, access the same data item, and at least one is a write. Scanning item A (accessed at steps 1,4,5,6,7) and item B (steps 2,3,8):

Conflict	**Edge**
R1(A)@1 → W2(A)@5	T1 → T2
R1(A)@1 → W3(A)@7	T1 → T3
R3(A)@4 → W2(A)@5	T3 → T2
W2(A)@5 → R4(A)@6	T2 → T4
W2(A)@5 → W3(A)@7	T2 → T3
R4(A)@6 → W3(A)@7	T4 → T3
R2(B)@2 → W1(B)@3	T2 → T1
R2(B)@2 → W4(B)@8	T2 → T4
W1(B)@3 → W4(B)@8	T1 → T4

Step 2 — Draw the precedence (serialization) graph

Nodes: T1, T2, T3, T4. Directed edges as derived above:

T1 → T2 T1 → T3 T1 → T4

T3 → T2 T2 → T3 T2 → T4

T4 → T3 T2 → T1

Step 3 — Check for a cycle

The edge T1 → T2 (from step 1) and the edge T2 → T1 (from step 8) together form a cycle T1 → T2 → T1.

Conclusion: Since the precedence graph contains a cycle, schedule S is NOT conflict-serializable. No serial order of T1..T4 produces an equivalent result; the system must either disallow this interleaving (via a scheduler such as 2PL) or roll one of the involved transactions back.

Q5. Basic Two-Phase Locking (2PL) & Deadlock (BL3)

Transactions on data items A and B:

T1: Lock-X(A); Read(A); Write(A); Lock-S(B); Read(B); Unlock(A); Unlock(B)

T2: Lock-X(B); Read(B); Write(B); Lock-S(A); Read(A); Unlock(B); Unlock(A)

Applying 2PL

Both transactions correctly follow 2PL: each has a growing phase (acquiring Lock-X, then Lock-S) followed by a shrinking phase (Unlock, Unlock) with no lock acquired after the first unlock — so 2PL's rule is respected by each transaction individually.

Interleaved execution and deadlock

Time	**T1**	**T2**
t1	Lock-X(A) — granted	
t2		Lock-X(B) — granted
t3	Read(A), Write(A)	
t4		Read(B), Write(B)
t5	Lock-S(B) — BLOCKED (T2 holds X on B)	
t6		Lock-S(A) — BLOCKED (T1 holds X on A)

T1 waits for T2 to release B, and T2 waits for T1 to release A — a circular wait, i.e. deadlock.

Detection and resolution

1. Build a wait-for graph: T1 → T2 (T1 waits for a lock held by T2) and T2 → T1 (T2 waits for a lock held by T1). The graph has a cycle T1 → T2 → T1, confirming deadlock.

2. The deadlock-detection component (or timeout mechanism) selects a victim, e.g. T2 (using a criterion such as "youngest transaction" or "least work done").

3. T2 is rolled back: its Lock-X(B) is released and all of T2's changes to B are undone.

4. T1's pending Lock-S(B) request is now granted; T1 completes and releases both locks. T2 is restarted from the beginning.

Q6. Wait-Die vs Wound-Wait Deadlock Prevention (BL4)

Timestamps: TS(T1) = 10 (older), TS(T2) = 20 (younger). Both request a lock currently held by the other.

Wait-Die scheme (non-preemptive: "old waits, young dies")

Requesting transaction	Rule applied	Outcome
T1 (older, TS=10) requests item held by T2 (younger)	older is allowed to wait for younger	T1 waits
T2 (younger, TS=20) requests item held by T1 (older)	younger requesting an item held by an older transaction must die	T2 is aborted and restarted with its original timestamp (10)

Wound-Wait scheme (preemptive: "old wounds, young waits")

Requesting transaction	Rule applied	Outcome
T1 (older) requests item held by T2 (younger)	older wounds (preempts) younger	T2 is rolled back immediately; T1 takes the lock
T2 (younger) requests item held by T1 (older)	younger transaction must wait for older	T2 waits

Comparison

- Wait-Die: transactions only ever wait for older ones; a younger transaction requesting from an older one is aborted (rolled back) — non-preemptive, may cause many restarts of young transactions.
- Wound-Wait: an older transaction preempts (wounds) a younger one holding a needed resource, forcing it to abort; a younger transaction simply waits for an older one — preemptive, tends to cause fewer restarts overall because wounded transactions are usually younger and have done less work.
- Both schemes are deadlock-free because they always allow only one direction of waiting (older→younger or younger→older, never both), so no wait-for cycle can form; restarted transactions keep their original timestamp to guarantee eventual progress (no starvation).

Q7. Immediate-Update Recovery Protocol (BL3)

Log (with a checkpoint) leading up to a system crash:

<T1 start>

<T1, A, 10, 20> -- old value 10, new value 20

<T2 start>

<T2, B, 30, 40>

<checkpoint {T1,T2}>

<T1 commit>

<T3 start>

<T3, C, 50, 60>

<T2, D, 70, 80>

-- SYSTEM CRASH --

Applying Immediate Update recovery

1. Because updates are applied to the database immediately (even before commit), any transaction that was active at the time of the crash may already have written its changes to disk — those changes must be UNDONE.

2. Scan the log backward from the crash point to the most recent checkpoint (and, for REDO, forward from the checkpoint) to classify every transaction as committed or active.

3. T1 has a <T1 commit> record after the checkpoint → REDO T1: reapply A = 20 (guarantees durability even if the page hadn't yet been flushed).

4. T2 has no commit record before the crash → UNDO T2: restore B = 30 and D = 70 (using the old values recorded in the log), then write a <T2 abort> record.

5. T3 has no commit record → UNDO T3: restore C = 50, then write a <T3 abort> record.

Undo operations are performed in reverse chronological order (most recent write undone first); redo operations are performed in forward chronological order, starting from the checkpoint.

Q8. Deferred-Update (NO-UNDO/REDO) Recovery (BL3)

Log file:

<T1 start>

<T2 start>

<T1, A, 20> -- deferred: value buffered, not yet written to DB

<T1 commit>

<checkpoint {T2}>

<T2, B, 40>

<T3 start>

<T3, C, 60>

<T2 commit>

-- SYSTEM CRASH -- (T3 never committed)

Applying Deferred Update recovery

1. Under NO-UNDO/REDO, a transaction's writes are held in a private buffer/log and only physically applied to the database AFTER it commits — so an uncommitted transaction's changes are never on disk, and UNDO is never required.

2. Scan the log forward from the last checkpoint. Build two lists: committed transactions and active (uncommitted) transactions at the time of the crash.

3. Committed after checkpoint: T2 (and T1, already committed before the checkpoint, needs no further action since a checkpoint guarantees its data was flushed). REDO T2's writes: reapply B = 40.

4. T3 has no commit record → its updates (C = 60) were never written to the database in the first place, so nothing needs to be undone; T3 is simply ignored/restarted later.

This is why the technique is called NO-UNDO/REDO: recovery only ever re-applies (REDO) the writes of transactions that committed after the last checkpoint; it never has to undo anything.

Q9. Concurrency Control Design — Strict 2PL for a Banking System (BL4)

Scenario: T1 transfers funds from account A to account B; T2 concurrently reads the balance of A (or B).

T1: Read(A); A := A - amt; Write(A); Read(B); B := B + amt; Write(B); Commit

T2: Read(A) [balance inquiry]

Design using Strict 2PL

1. Every read acquires a shared (S) lock and every write acquires an exclusive (X) lock, following ordinary 2PL: all locks for a transaction are acquired before any is released (growing phase, then shrinking phase).

2. Strict 2PL adds the rule that ALL locks — shared and exclusive — are held until the transaction COMMITS or ABORTS (not released as soon as an operation finishes). This is the key design choice for the banking scenario.

3. T1 sequence: Lock-X(A) → Read/Write(A) → Lock-X(B) → Read/Write(B) → Commit → Unlock(A), Unlock(B) (both released together, only at commit).

4. T2 requests Lock-S(A) to read the balance. If T2's request arrives while T1 still holds Lock-X(A) (i.e., T1 has not yet committed), T2 is blocked and must wait.

5. T2 is only granted the lock, and allowed to read A, after T1 commits — so T2 can never see the intermediate, partially-updated (dirty) state where money has left A but not yet arrived in B.

Why strict 2PL (rather than basic 2PL)

- Recoverability & cascadelessness: because locks are held until commit, no other transaction can read a value written by an uncommitted transaction, so T2 can never read dirty data from T1 and cascading rollbacks are impossible.
- Serializability: like basic 2PL, strict 2PL guarantees conflict-serializable schedules, but it additionally guarantees the schedule is equivalent to a serial order that respects commit order — appropriate for a financial system where audit consistency matters.
- Deadlock handling: since T1 acquires locks on both A and B, a deadlock could occur if another transaction locks B first and then requests A; the DBMS applies deadlock detection (wait-for graph) or a prevention scheme (e.g. Wound-Wait) and rolls back a victim transaction as in Q5/Q6.

Q10. Pseudocode for Heuristic Query Optimization (BL4)

```
function optimizeQuery(queryTree):
  // Rule 1: break conjunctive selections into individual selections
  queryTree = splitConjunctiveSelections(queryTree)

  // Rule 2: push each selection down as far as the tree allows
  for each selection node σ_cond in queryTree:
```

```
    while σ_cond can commute with its child node (join/product/projection):
        move σ_cond below its child
    end while
end for

// Rule 3: project early -- push projections toward the leaves
for each leaf relation R in queryTree:
    attrs = attributesNeededFrom(R, queryTree)   // used later + in predicates
    insert π_attrs immediately above R
end for

// Rule 4: convert (selection + Cartesian product) into a join
for each × node with a matching equality selection above it:
    replace (σ_join_cond(A × B)) with (A ⋈_join_cond B)
end for

// Rule 5: order joins by estimated size of intermediate result
joinList = extractAllJoinNodes(queryTree)
for each pair (Ji, Jj) in joinList:
    estimate size(Ji), size(Jj) using selectivity factors
end for
sort joinList ascending by estimated intermediate size
rebuild left-deep join tree following sorted order
    (smallest-result joins executed first, feeding larger joins)

// Rule 6: identify and reuse common sub-expressions
reuse any repeated sub-tree instead of recomputing it

return queryTree   // heuristically optimized plan
```

This pseudocode encodes the standard heuristic rules used by relational query optimizers: selections are performed as early as possible to shrink relations before expensive operations; projections are performed early to reduce tuple width; Cartesian products are combined with selections to become joins; and joins

are ordered so that the join producing the smallest intermediate relation runs first, minimizing the size (and I/O cost) of subsequent operations.